



Module 3 – Installation

Choisir, installer et vérifier un cluster
Kubernetes – Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous serez capable de :

- **choisir** la bonne solution d'installation ;
- installer **kubectl** et vérifier son intégrité ;
- lire et manipuler un fichier **kubeconfig** ;
- créer un cluster de poste avec **minikube** ou **kind** ;
- décrire un bootstrap **kubeadm** : init, join, CNI, certificats ;
- **vérifier** qu'un cluster fraîchement installé est sain.



Plan du module

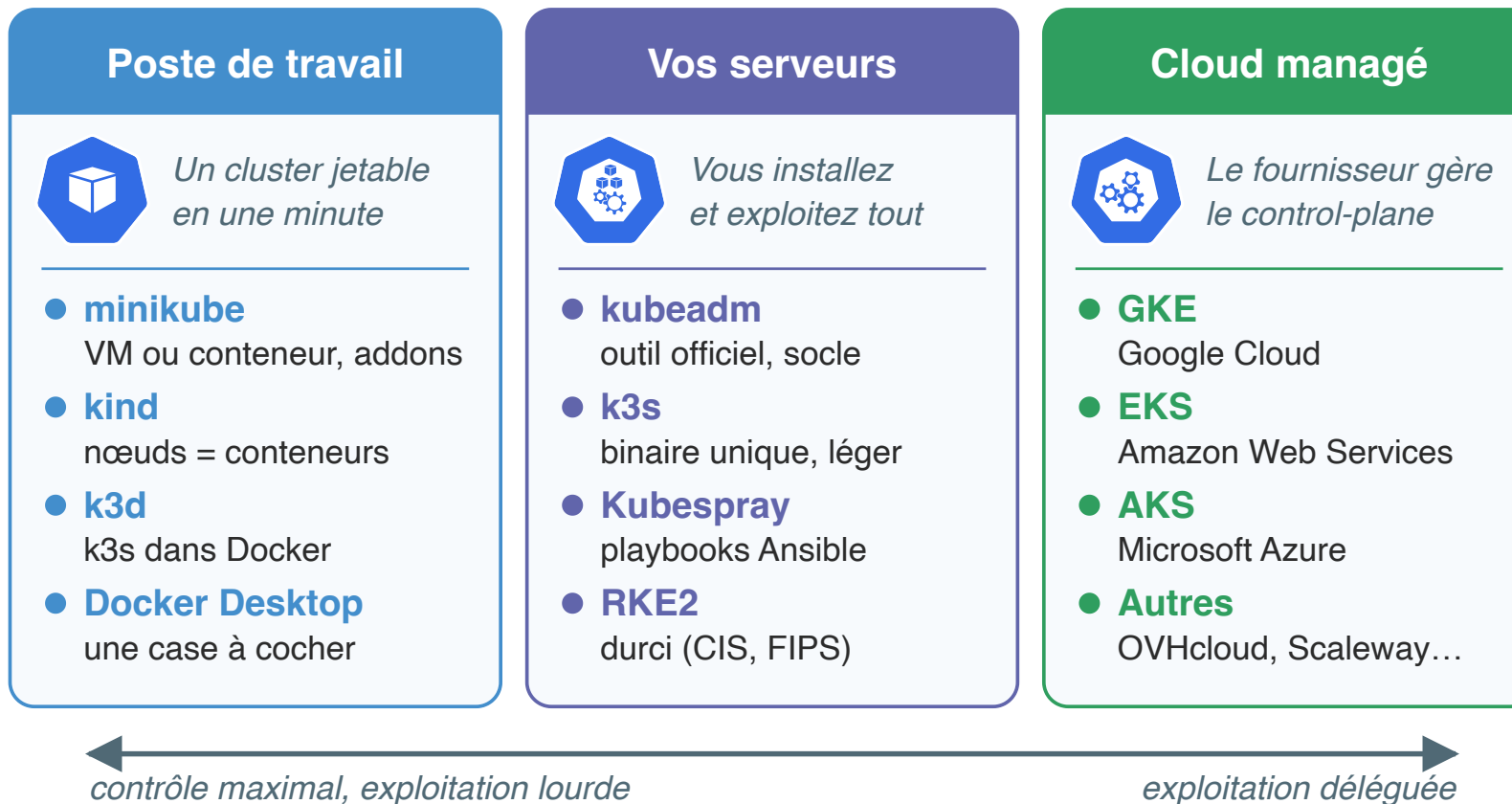
1. **Panorama** : poste de travail, serveurs, cloud managé.
2. **Les outils du poste** : kubectl, kubeconfig, minikube, kind.
3. **kubeadm** : prérequis, init, join, certificats, mise à niveau, vérifications.

Durée : 1 h 15, dont la démo 3 (kubeadm et kubeconfig) et le TP 3 (préparer les accès de l'équipe).



1. Panorama des solutions

Trois familles d'installation



Le poste de travail et les serveurs

Poste de travail	Principe
minikube	VM ou conteneur, add-ons
kind	Kubernetes IN Docker
k3d	k3s dans Docker
Docker Desktop	1 nœud intégré

Serveurs	Principe
kubeadm	outil officiel, brique de base
k3s	distribution légère (SUSE)
Kubespray	kubeadm piloté par Ansible
RKE2	distribution durcie (SUSE)

Sur le poste, le cluster est **jetable** : on le recrée plutôt que de le réparer. Sur les serveurs, il **dure** : sauvegardes, certificats et mises à niveau deviennent votre métier.

Qui gère quoi ?

Qui s'en occupe ?	kind, minikube	kubeadm	GKE, EKS, AKS
 Applications	Vous	Vous	Vous
 Nœuds workers	L'outil	Vous	Partagé
 Control-plane	L'outil	Vous	Fournisseur
 etcd, sauvegardes	Jetable	Vous	Fournisseur
 Certificats, upgrades	On recrée	Vous	Fournisseur

Partagé : vous gérez les pools de nœuds, le fournisseur fournit images et mises à jour.

Matrice de choix

Solution	Multi-nœuds	Production	Installation	Usage typique
minikube			faible	Dev local, addons
kind			faible	CI, tests, formation
k3d			faible	k3s en conteneurs
k3s			faible	Edge, IoT, petits sites
kubeadm			élevé	Socle, maîtrise totale
Kubespray			moyen	Parc de VM, Ansible
RKE2			moyen	Secteurs régulés
GKE, EKS, AKS			faible	Production déléguée

 oui
  partiel (--nodes)
  non

Installation : effort de mise en place

2. Les outils du poste

kubectl, kubeconfig, minikube, kind

Installer kubectl sous Linux x86_64

```
# 1. Télécharger le binaire ET son empreinte (même version que le cluster)
curl -LO "https://dl.k8s.io/release/v1.35.0/bin/linux/amd64/kubectl"
curl -LO "https://dl.k8s.io/release/v1.35.0/bin/linux/amd64/kubectl.sha256"
# 2. Vérifier l'intégrité AVANT d'installer
echo "$(cat kubectl.sha256) kubectl" | sha256sum --check
kubectl: OK
# 3. Installer et contrôler
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Une empreinte fausse donne `kubectl: FAILED` et un code retour non nul : on n'installe pas. Sur ARM64, remplacer `amd64` par `arm64`.

Installer kubectl sous macOS et Windows

macOS (Homebrew ou binaire)

```
brew install kubectl  
# ou binaire versionné :  
curl -LO "https://dl.k8s.io/release/  
v1.35.0/bin/darwin/arm64/kubectl"
```

darwin/amd64 sur Mac Intel.

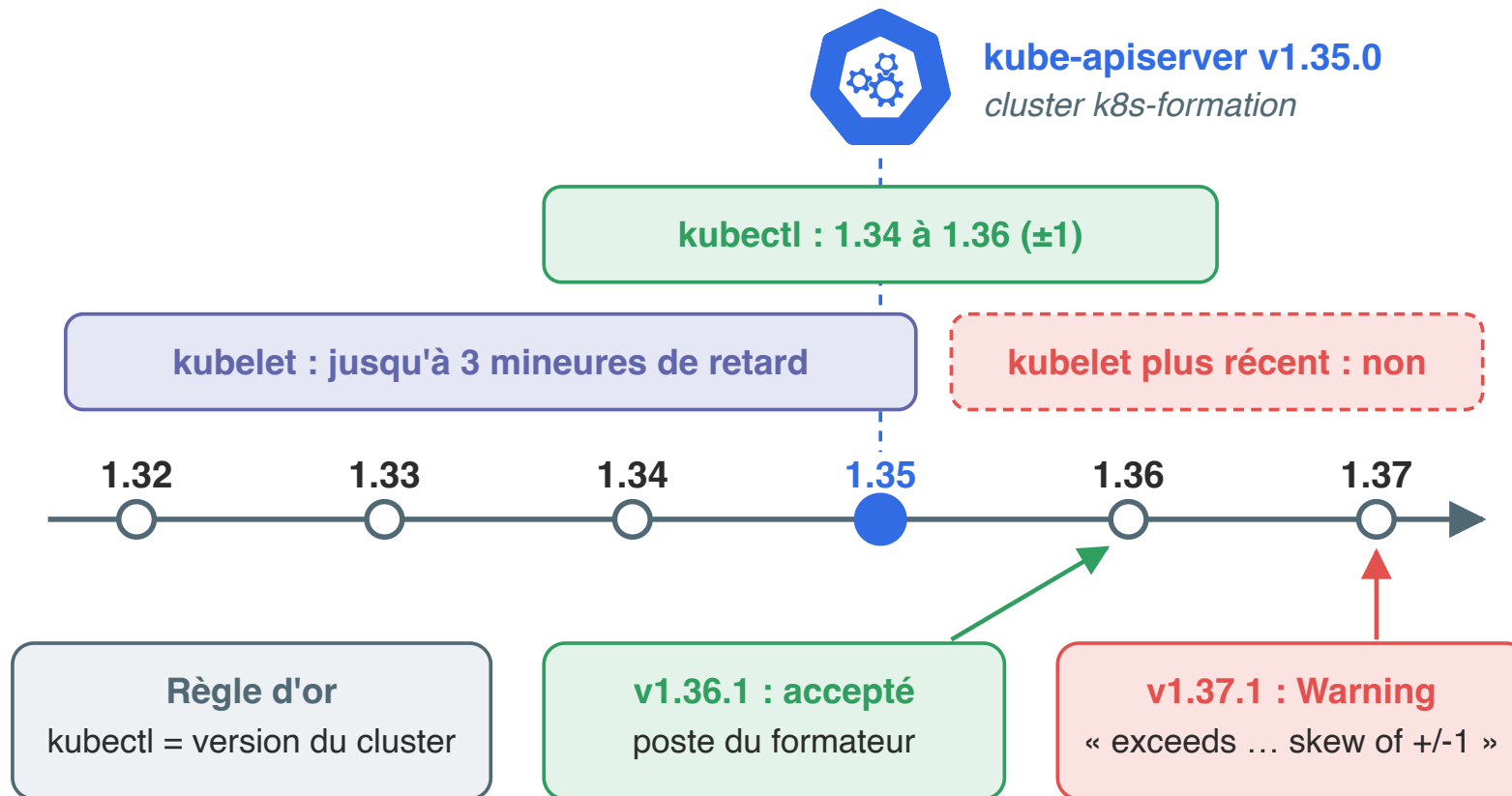
Windows (PowerShell)

```
winget install -e --id Kubernetes.kubectl  
# ou : choco install kubernetes-cli  
kubectl version --client
```

Sous WSL2, suivre la procédure Linux.

Les gestionnaires de paquets installent la **dernière** version (1.37 le jour du test), pas forcément celle du cluster.

Compatibilité de version : ± 1 mineure

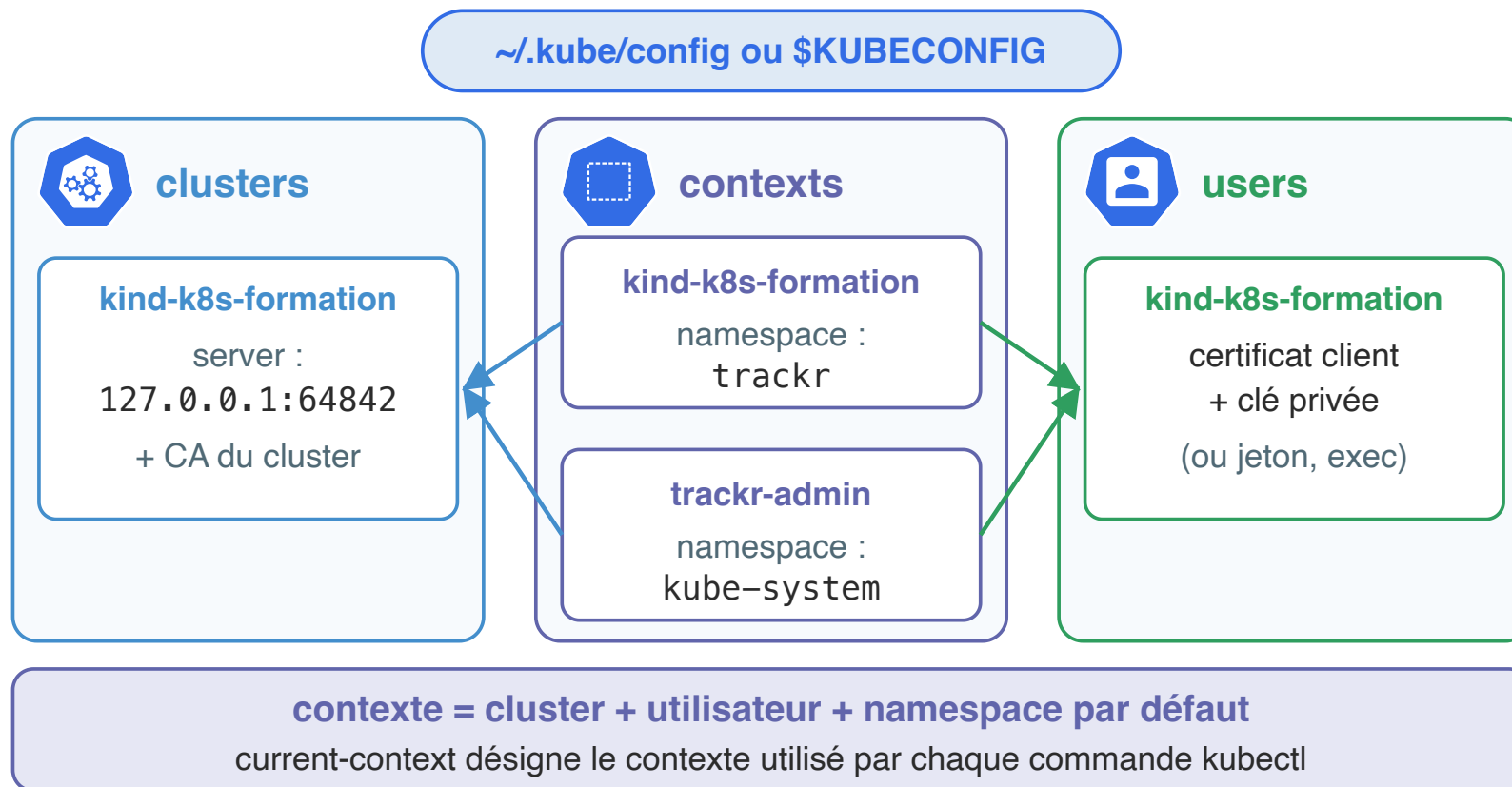


Complétion, alias et plugins

```
# bash (Linux, WSL2)
source <(kubectl completion bash)
echo 'source <(kubectl completion bash)' >> ~/.bashrc
# zsh (macOS)
echo 'source <(kubectl completion zsh)' >> ~/.zshrc
# PowerShell
kubectl completion powershell | Out-String | Invoke-Expression
# alias court, avec complétion
alias k=kubectl
complete -o default -F __start_kubectl k
```

krew est le gestionnaire de plugins de kubectl : `kubectl krew install ctx ns` ajoute `kubectl ctx` et `kubectl ns` pour changer de contexte ou de namespace (mention, non utilisé dans ce cours).

Le fichier kubeconfig



Le kubeconfig du lab

```
$ kubectl config view --minify
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://127.0.0.1:64842
  name: kind-k8s-formation
contexts:
- context:
    cluster: kind-k8s-formation
    user: kind-k8s-formation
  name: kind-k8s-formation
current-context: kind-k8s-formation
```

- `--minify` : seulement le contexte courant ; les clés sont masquées (`DATA+OMITTED`).
- Le port **64842** est tiré au hasard par kind : il change d'un poste à l'autre.
- L'utilisateur s'authentifie par **certificat client** signé par la CA du cluster.
- Ce fichier donne les pleins pouvoirs : à protéger comme une clé SSH.

kubectl config au quotidien

```
$ kubectl config get-contexts
$ kubectl config set-context --current --namespace=trackr
Context "kind-k8s-formation" modified.
$ kubectl config set-context trackr-admin --cluster=kind-k8s-formation \
  --user=kind-k8s-formation --namespace=kube-system
Context "trackr-admin" created.
$ kubectl config use-context trackr-admin
Switched to context "trackr-admin".
$ kubectl config rename-context trackr-admin lab-kube-system
$ kubectl config delete-context lab-kube-system
$ KUBECONFIG=~/.kube/config:./autre.kubeconfig kubectl config get-contexts
```

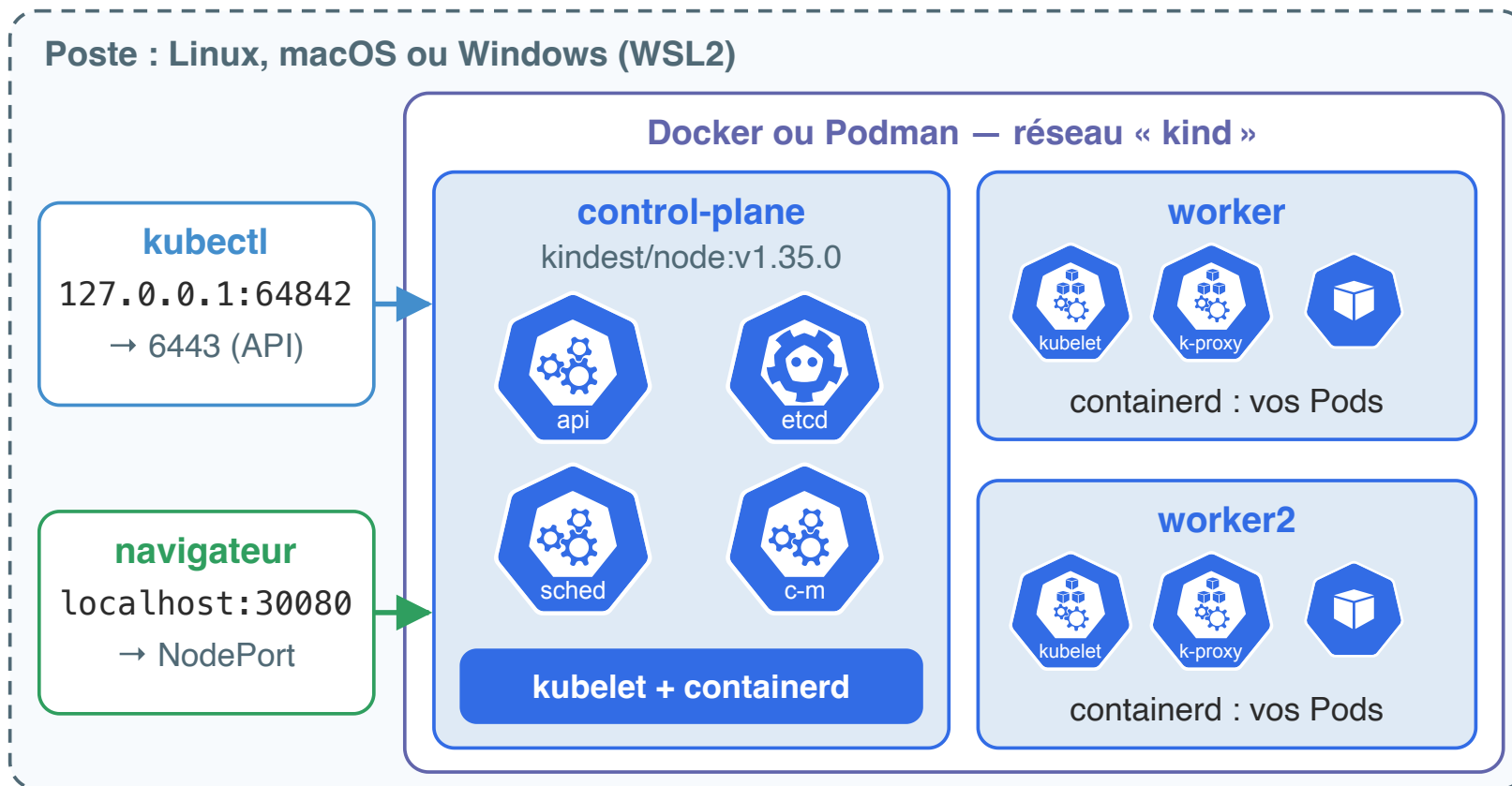
Plusieurs fichiers se **fusionnent** via `KUBECONFIG` (séparateur `:` ; `;` sous Windows). Toujours vérifier `kubectl config current-context` avant une action destructive.

minikube : drivers, addons, dashboard

```
$ minikube start --driver=podman --container-runtime=containerd --kubernetes-version=v1.35.0
* Using rootless Podman driver
* Done! kubectl is now configured to use "m03-minikube" cluster and "default" namespace by default
$ minikube status
host: Running
kubelet: Running
apiserver: Running
$ minikube addons enable dashboard
$ minikube dashboard --url
http://127.0.0.1:54763/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
$ minikube delete
```

- **Drivers :** `docker`, `podman`, `kvm2`, `hyperv`, `qemu` ... (conteneur ou VM). Premier démarrage : **5 min 51 s** (image de base de 470 Mo).
- Rejoué avec le profil `-p m03-minikube`, omis ici pour la lisibilité.

kind : un nœud = un conteneur

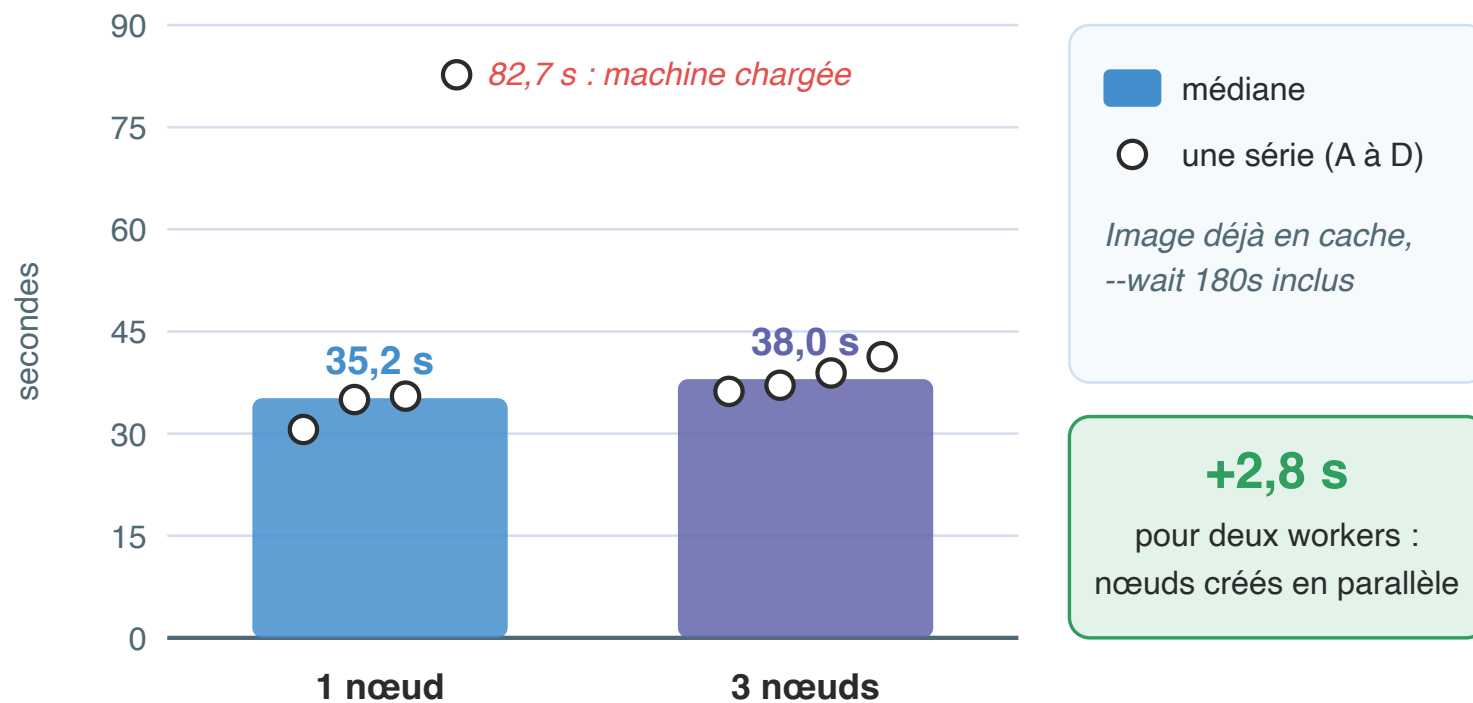


Notre lab : kind-config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k8s-formation
nodes:
  - role: control-plane
    image: kindest/node:v1.35.0
    extraPortMappings:
      - containerPort: 30080
        hostPort: 30080
  - role: worker
    image: kindest/node:v1.35.0
  - role: worker
    image: kindest/node:v1.35.0
```

- **Un rôle par nœud** : 1 control-plane, 2 workers.
- **Image figée** : la version de Kubernetes est celle de l'image `kindest/node`.
- **extraPortMappings** publie le NodePort 30080 sur le poste.
- `code/lab/lab-up.sh` crée le cluster, puis installe **metrics-server** (patch idempotent).
- Podman : `LAB_PROVIDER=podman ./lab-up.sh`.

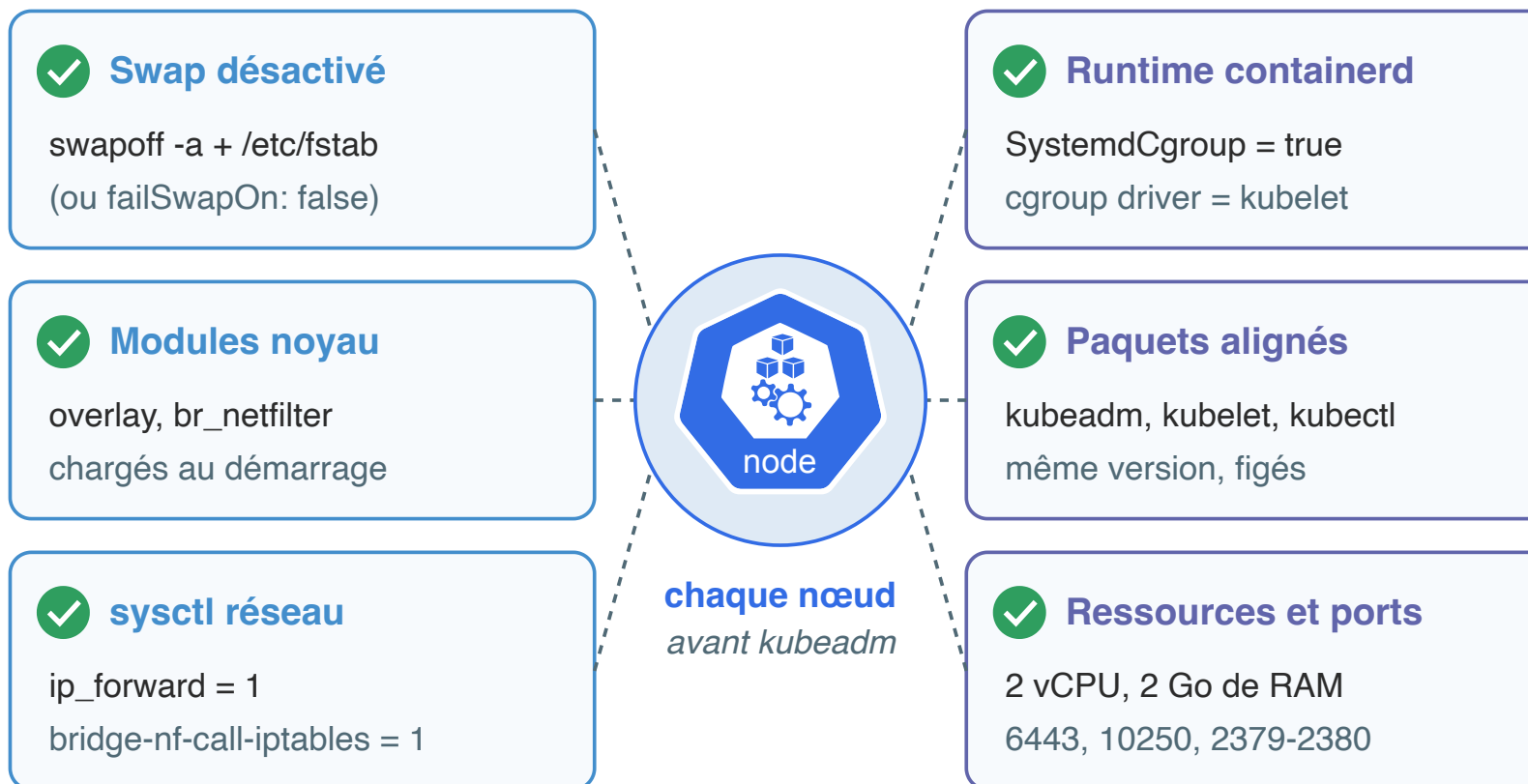
Créer un cluster kind : 1 ou 3 nœuds



Mesuré sur le lab : kind v0.31, Podman rootless (4 vCPU, 6 Go), quatre séries par taille.

3. kubeadm : installer sur des serveurs

Préparer chaque nœud

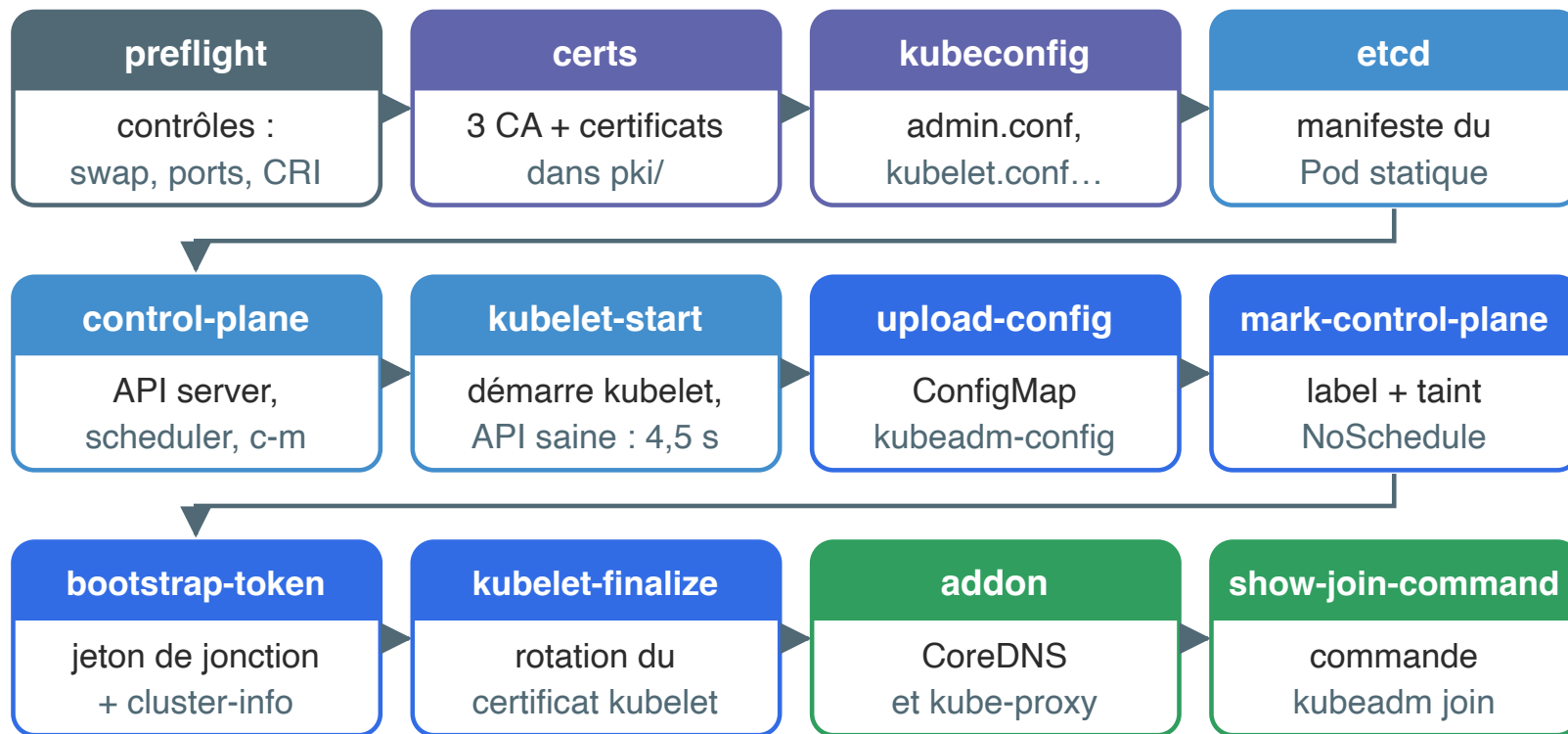


Préparer le nœud : les commandes

```
sudo swapoff -a && sudo sed -i '/ swap / s/^/#/' /etc/fstab
printf 'overlay\nbr_netfilter\n' | sudo tee /etc/modules-load.d/k8s.conf
sudo modprobe overlay && sudo modprobe br_netfilter
printf 'net.ipv4.ip_forward = 1\nnet.bridge.bridge-nf-call-iptables = 1\n' \
| sudo tee /etc/sysctl.d/k8s.conf && sudo sysctl --system
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo apt-mark hold kubelet kubeadm kubectl
```

Documentation officielle (Debian/Ubuntu), **non rejouée sur le lab** : le nœud kind est un conteneur déjà préparé. On y vérifie le résultat : swap vide, `ip_forward = 1`, `SystemdCgroup = true`.

kubeadm init : les phases



Mesuré sur le cluster jetable : kubeadm init complet en 8,0 s

kubeadm init : la sortie réelle

```
# kubeadm init --config /kind/kubeadm.conf
[init] Using Kubernetes version: v1.35.0
[certs] Generating "ca" certificate and key
[kubeconfig] Writing "admin.conf" kubeconfig file
[etcd] Creating static Pod manifest for local etcd in "/etc/kubernetes/manifests"
[control-plane] Creating static Pod manifest for "kube-apiserver"
[control-plane-check] kube-apiserver is healthy after 4.501628067s
[mark-control-plane] Marking the node m03-jetable-control-plane as control-plane by adding the taints [...]
[addons] Applied essential addon: CoreDNS
Your Kubernetes control-plane has initialized successfully!
real    0m8.027s
```

Rejoué sur le cluster jetable après `kubeadm reset -f`. Sur un serveur :

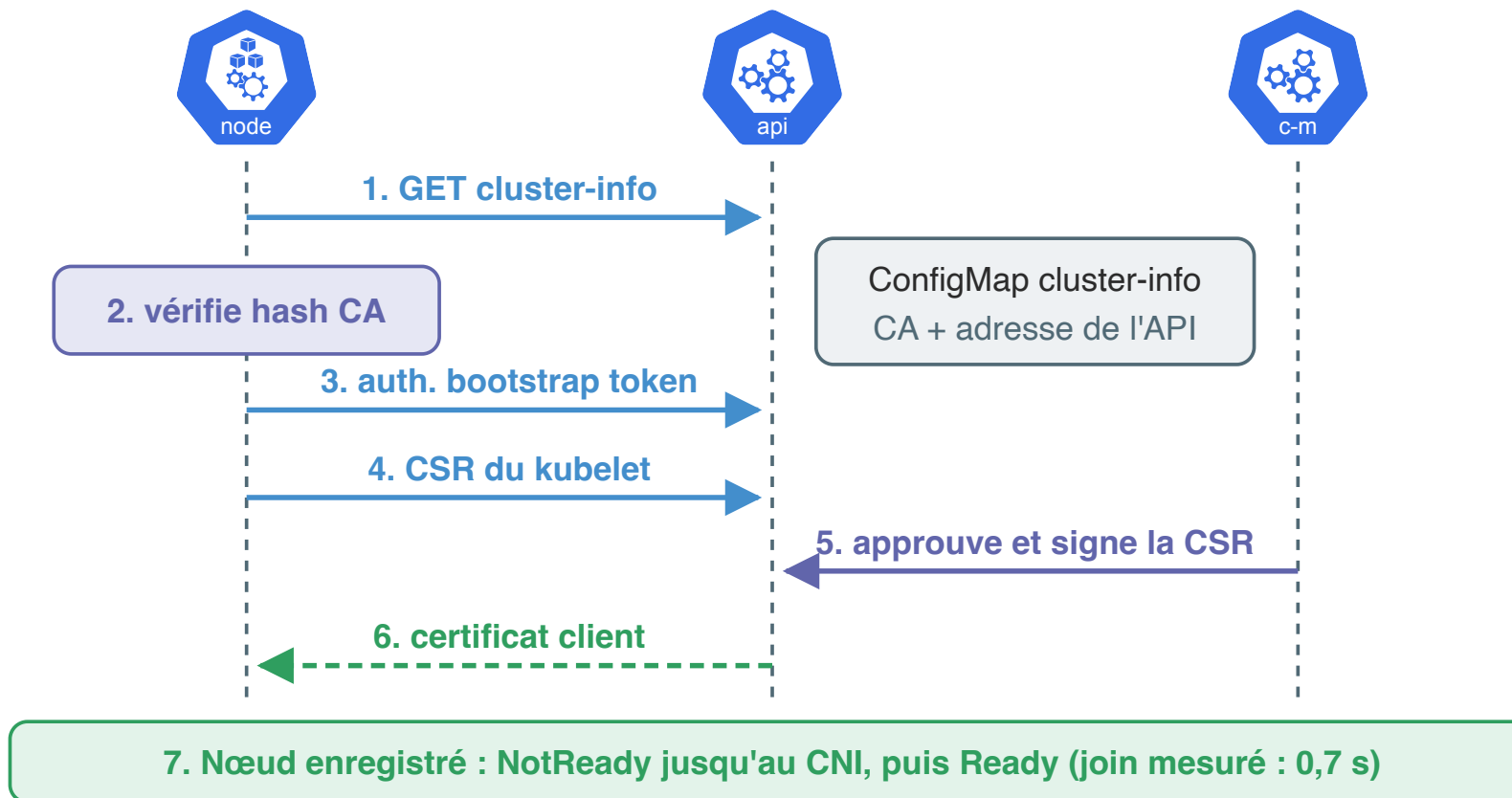
```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16.
```

Installer le réseau de Pods (CNI)

```
# kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
m03-jetable-control-plane          NotReady  control-plane  14s   v1.35.0
# kubectl describe node m03-jetable-control-plane | grep Ready
Ready  False ... cni plugin not initialized
# kubectl apply -f kindnet.yaml
daemonset.apps/kindnet created
# kubectl wait --for=condition=Ready nodes --all --timeout=180s
node/m03-jetable-control-plane condition met
node/m03-jetable-worker condition met
```

kubeadm n'installe aucun CNI : sans lui, nœuds `NotReady` et CoreDNS `Pending`. Calico, Cilium, Flannel sont courants ; le lab utilise kindnet.

kubeadm join : le parcours

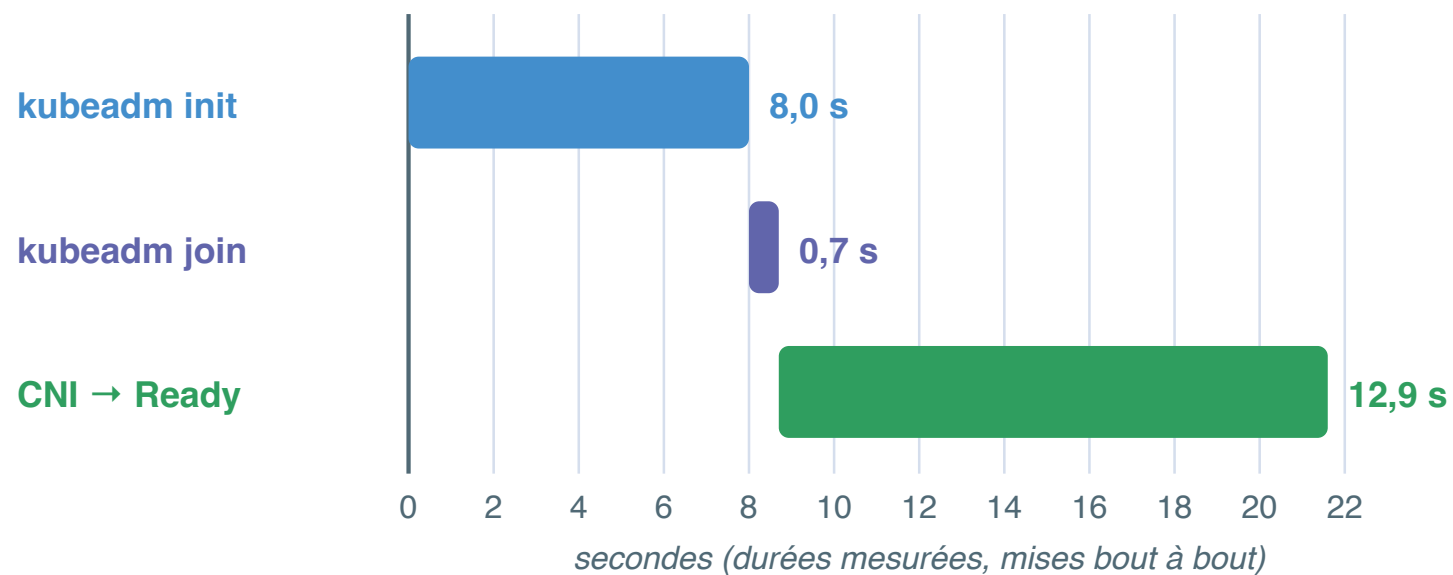


Jeton et commande de jonction

```
# kubeadm token list
TOKEN          TTL    EXPIRES          USAGES          ...
abcdef.0123456789abcdef  23h    2026-10-05T20:36:53Z  authentication, signing  ...
# kubeadm token create --print-join-command
kubeadm join k8s-formation-control-plane:6443 --token fsznyr.taeu721pn346xhd5 \
  --discovery-token-ca-cert-hash sha256:e9a5a21cf45acf3ae5906c97118f6a8c15ab34d526859469e379dfe34e291bc6
# kubeadm token delete fsznyr
bootstrap token "fsznyr" deleted
```

- Un jeton vit **24 h** : au-delà, on en recrée un avec `--print-join-command`.
- Le **hash** épingle la CA : le nœud refuse un faux control-plane.

Chronologie mesurée d'un bootstrap

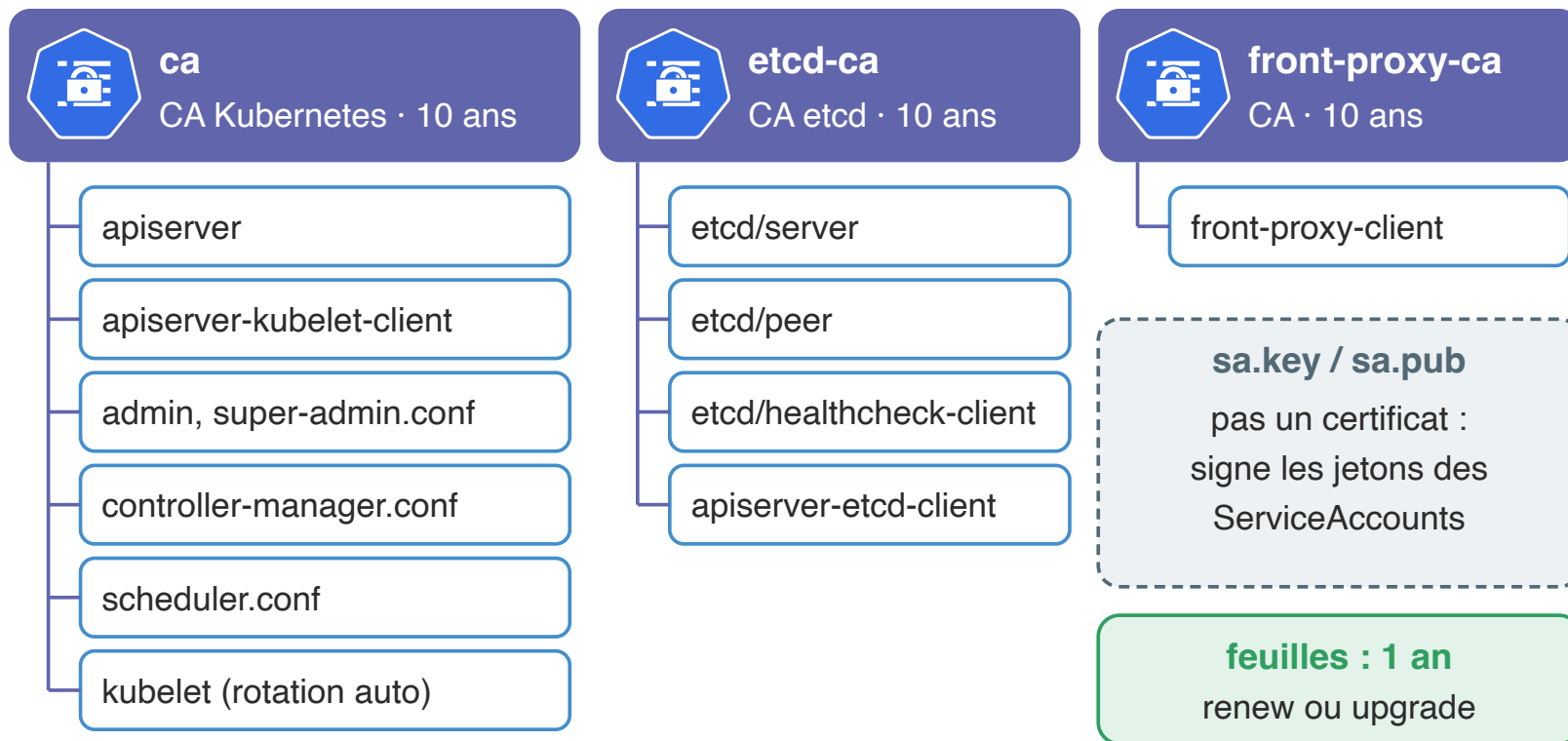


≈ 21,6 s d'outillage pour deux nœuds Ready (cluster jetable)

Dans l'init, kube-apiserver déclaré sain après 4,5 s d'attente

Mesuré sur le cluster jetable m03-jetable (2 nœuds kind, Kubernetes v1.35.0).

La PKI du cluster



Tout est dans /etc/kubernetes/pki ; les fichiers .conf embarquent leur certificat client.

Certificats et mises à niveau

```
# kubeadm certs check-expiration
CERTIFICATE          EXPIRES              RESIDUAL TIME    CERTIFICATE AUTHORITY
admin.conf           Oct 04, 2027 20:36 UTC 364d             ca
...
ca                   Oct 01, 2036 20:36 UTC 9y
# kubeadm upgrade plan
[upgrade/versions] Cluster version: 1.35.0
[upgrade/versions] Target version: v1.35.9
kubeadm upgrade apply v1.35.9
```

- `kubeadm certs renew all` renouvelle les feuilles (testé sur le cluster jetable) ; `upgrade apply` aussi.
- Mise à niveau **une mineure à la fois** : control-plane d'abord, puis kubelets nœud par nœud.

Vérifier l'installation

1		Nœuds Ready	<code>kubectl get nodes</code>
2		Pods système Running	<code>kubectl get pods -n kube-system</code>
3		API saine	<code>kubectl get --raw='/readyz'</code>
4		DNS du cluster	<code>nslookup <svc>.<ns>.svc.cluster.local</code>
5		Pod vers Service	<code>wget -q0- http://trackr-smoke:9898</code>
6		Droits de l'admin	<code>kubectl auth can-i '*' '*'</code>
Six contrôles verts : le cluster est prêt pour TrackR			

Test de fumée TrackR

```
$ kubectl apply -n trackr -f smoke-test.yaml
pod/trackr-smoke created
service/trackr-smoke created
$ kubectl -n trackr run dns-check --image=busybox:1.37 --rm -i --restart=Never \
  -- nslookup trackr-smoke.trackr.svc.cluster.local
Name:   trackr-smoke.trackr.svc.cluster.local
Address: 10.96.168.133
$ kubectl -n trackr run http-check --image=busybox:1.37 --rm -i --restart=Never \
  -- wget -qO- http://trackr-smoke:9898/api/info
"message": "greetings from podinfo v6.9.2",
```

Un Pod, un Service, une résolution DNS, une requête HTTP : la chaîne complète **scheduler, CNI, kube-proxy, CoreDNS** est validée.

Démo et TP

Démo 3 – kubeadm et kubeconfig

`demos/demo-03-kubeadm-kubeconfig.md`

kubeadm en lecture sur le lab, `admin.conf` contre kubeconfig de l'hôte, puis init, join et CNI sur un cluster jetable.

TP 3 – Préparer les accès de l'équipe

`tp/tp-03-installer-cluster.md`

Ticket Logivia : un kubeconfig d'équipe à deux contextes, un rapport de santé, votre test de fumée et deux questions kubeadm.

À retenir

- Poste : **kind** ou **minikube** ; serveurs : **kubeadm** et ses dérivés ; cloud : **GKE, EKS, AKS**.
- kubectl à **±1 mineure** du cluster, **empreinte vérifiée**.
- Un **contexte** = cluster + utilisateur + namespace.
- kubeadm n'installe **ni le runtime ni le CNI**.
- Certificats feuilles : **1 an**, à surveiller.
- Toujours finir par un **test de fumée**.

